

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA
SEMESTER III TAHUN AJARAN GANJIL 2025/2026**



IF350 – SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

Minggu/Pertemuan ke 8 – Scheduling and Budgeting

Eunike Endariahna Surbakti, S.Kom., M.T.I

Weekly Learning Outcomes of the Course (Sub-CLO)



Sub-CLO 0713 Work effectively in teams with an Agile approach and complete project deliverables on time (C5)

Learning Objectives



Objective: To analyze and apply modern techniques (2020-2025) for managing schedule and budget in complex software projects.

Key Focus Areas:

Emerging project management trends (AI, Hybrid Methods).

Best practices for time and cost estimation.

Advanced control metrics (EVM, Burn Rate).

Case study analysis of real-world budget/schedule challenges.

Target Audience: Future Software Project Managers (Focus on practical, actionable knowledge).

The Landscape 2020-2025



- AI Integration: Generative AI is increasingly used for initial estimation, resource allocation, and daily task automation (forecasting, reporting).
- Hybrid Models: Pure Waterfall is rare; most large projects use hybrid models (Agile development within a structured program framework).
- Remote/Hybrid Work: Requires new scheduling and budgeting considerations for dispersed teams and asynchronous communication.
- Data-Driven Control: Increased demand for *real-time* data (not just status reports) for stringent control and rapid decision-making.
- Value Focus: Budgets are increasingly tied to Time-to-Market (TTM) and Return on Investment (ROI), not just cost variance.

Why Schedule and Budget Control Matters

- Statistics (Post-2020): Approximately 70% of projects still fail to meet initial scope, time, *or* budget targets. (Source: Project Management Institute, various reports)
- The Iron Triangle in Software:
Scope: Must be managed continuously (often variable in Agile).
Time (Schedule): Directly impacts market opportunity (TTM).
Cost (Budget): Directly impacts profitability and viability.
- Key Challenge: Balancing the need for predictability (traditional need) with the need for adaptability (modern software need).

Agile Dominance and Hybridization



Agile Methodology: The standard for software development since 2020. Focuses on iterative delivery, adapting to change, and working software over documentation. **Schedule Control:** Fixed duration (Sprints/Iterations), variable scope (feature flexibility).

Hybrid Project Management: The prevailing model for enterprise software. **Definition:** Combining a predictive approach (e.g., initial high-level planning, fixed budget) with an adaptive approach (e.g., Agile development for execution). **Benefit:** Provides organizational *predictability* for stakeholders while enabling team *flexibility* for developers.

Scheduling in Agile: Sprints and Velocity

Sprints/Iterations: Fixed, short timeboxes (typically 1-4 weeks).
The schedule is fixed; the scope is flexible.

Velocity: The average amount of work (usually measured in Story Points) a team completes per Sprint.

Purpose: The primary tool for forecasting future schedules.

$$\text{Remaining Sprints} = \frac{\text{Total Estimated Story Points}}{\text{Team Average Velocity}}$$

Critical Practice: Velocity must be tracked consistently (at least 3-5 sprints) before being used for reliable forecasting.

The Role of Automation and AI in Scheduling

- Generative AI (2023+ Trend): Used to generate initial task breakdowns from high-level user stories, reducing initial WBS (Work Breakdown Structure) planning time.
- Predictive Scheduling: Machine learning models analyze historical project data (past tasks, resource types, dependencies) to:
 - Suggest optimal task sequencing (beyond simple Critical Path).
 - Identify high-risk dependencies that could cause schedule slippage.
 - Benefit: Enables real-time risk adjustment of the timeline.

Impact of Remote/Hybrid Work on Schedule

- The Challenge: Time zone differences, communication lag, and loss of "osmotic communication."
- Schedule Mitigation Strategies:
 - Asynchronous-First Planning: Document decisions clearly (e.g., Confluence, Jira) rather than relying solely on meetings.
 - Overlapping Hours: Schedule critical meetings only during maximum team overlap.
 - Increased Time for Integration: Add specific "integration/merging" time at the end of sprints, as these tasks often suffer most from hybrid work fragmentation.

Practice: Timeboxing and Iterative Planning

- Timeboxing: Assigning a fixed, non-negotiable duration to an activity (e.g., a "spike" for R&D, a Sprint). Benefit: Prevents Parkinson's Law (work expanding to fill the time allotted) and forces prioritization.
- Rolling Wave Planning: Detailed planning is done only for the near-term phase (e.g., the next 2-3 Sprints), while future phases are planned at a high level. Benefit: Reduces waste from detailed planning on distant scope that is likely to change.

Zero-Based Budgeting (ZBB) in IT Projects

- Traditional: Incremental budgeting (Last year's budget + X%).
- Value-Driven Costing: Prioritizing and funding features based on their potential ROI (Return on Investment) or CoD (Cost of Delay). Concept: Only the highest-value features are funded first, ensuring the biggest bang for the buck is delivered early.
- Zero-Based (ZBB): Every dollar must be justified from a "zero" base. Application: Requires the project manager to justify every cost based on its direct contribution to the project's value delivery.

Beyond Estimation: Focus on Forecasting

- Estimation (Pre-Project): Guessing the future based on past data and assumptions. (Example: COCOMO, function points).
- Forecasting (In-Progress): Calculating the future based on current performance data.
 - Crucial Metrics: EAC (Estimate At Completion) and ETC (Estimate To Complete).
 - Modern Requirement: Budget systems must update forecasts *automatically* based on real-time task completion and spend tracking.

Total Cost of Ownership (TCO) in Software Projects

- Project Budget (Initial): Focused on design, development, and testing.
- TCO (Life Cycle Cost): The full cost of acquiring, developing, operating, maintaining, and disposing of a software product over its lifespan (typically 5-10 years).
- Key Components (Post-2020):
 - Cloud Costs: Hosting, data storage, scaling fees (often a major budget overrun source).
 - Cybersecurity & Compliance: Audits, licensing for security tools.
 - Maintenance & Support: Often 15-20% of the initial development cost *per year*.
 - PM Mandate: Software PMs must now estimate and manage TCO, not just the development budget.

Budgeting in Agile (Fixed Time/Scope/Cost)

- The "Iron Triangle Flip": Agile often holds Time (Sprint Length) and Cost (Team Size) fixed, making Scope the primary variable.
- Fixed-Price Contracts: If a budget is fixed, Agile teams must manage scope through MoSCoW Prioritization (Must Have, Should Have, Could Have, Won't Have).
 - The "Won't Have" (out-of-scope) list is the primary budget defense mechanism.
- Burn Rate: The rate at which project funds are spent over time. The fundamental control mechanism for an Agile budget.

Contingency and Risk-Adjusted Budgeting

- Contingency Reserve: Funds allocated to known, accepted risks (e.g., "If integration with Legacy System X fails, we need Z budget"). Budgeting Rule: Typically 10-20% of the base estimate.
- Management Reserve: Funds allocated for *unknown-unknown* risks (e.g., a global pandemic, a sudden regulatory change). Control: Only the executive management controls this reserve.
- Risk-Adjusted Estimate: Incorporating the expected monetary value (EMV) of identified risks directly into the base estimate.

$$\text{EMV} = \text{Probability} \times \text{Impact Cost}$$

Key Budgeting Metrics

Cost Performance Index (CPI): Measures the financial efficiency of the project.

- $$CPI = \frac{\text{Earned Value (EV)}}{\text{Actual Cost (AC)}}$$

- $CPI < 1$: Over budget (Cost of work completed is greater than planned).

Budget Variance (CV): The difference between the value of work completed and the actual cost.

- $$CV = \text{Earned Value (EV)} - \text{Actual Cost (AC)}$$

To-Complete Performance Index (TCPI): The cost performance required for the remaining work to meet the original budget.

Practice: Transparent Cost Breakdown Structure (CBS)

- Requirement: A Cost Breakdown Structure must align directly with the WBS (Work Breakdown Structure).
- Modern CBS Categorization:
 - Direct Labor: Internal salaries, contractor rates, burden.
 - Indirect Costs: Project management overhead, training, facility costs.
 - Capital/Technology: Software licenses, cloud consumption, infrastructure (CaPex).
 - Risk/Contingency: Dedicated reserve funds.
- Benefit: Granular tracking allows stakeholders to see *exactly* where the money is going and enables quicker adjustments to high-spend areas.

Advanced Control & Emerging Trends



Earned Value Management (EVM) 2.0

EVM (Traditional): A powerful integration of scope, time, and cost. EV is traditionally calculated based on physical completion percentage.

Challenges in Software: How do you define "50% complete" for a piece of code?

EVM 2.0 / Fuzzy EVM:

- Adapting EVM for Agile by assigning EV based on completed Story Points (using Velocity as the baseline).
- Using predictive modeling (fuzzy logic) to estimate completion uncertainty, providing a range of possible end-dates and costs, not just a single point estimate

Data-Driven Decision Making (2020s Trend)

- Shift: Moving from anecdotal status reports ("We're on track") to objective metrics ("Our CPI is 0.92, indicating a projected 8% overrun").
- Key Data Tools: Business Intelligence (BI) and Data Visualization tools (e.g., Power BI, Tableau) are integrated with PM software (e.g., Jira, Asana).
- Mandate for PMs: Project Managers must possess strong data analysis skills to interpret metrics and present actionable insights to steering committees.

Focus on Strategic Alignment and Value (2020-2025)

Problem: Historically, projects were successful if they met *budget/schedule*, regardless of business outcome.

Modern View: A project is only successful if it meets its Strategic Business Objectives.

Metrics: Must track project metrics alongside business outcome metrics:

- Project Metric: SPI = 1.0 (On schedule).
- Business Metric: TTM reduced by 15%, Revenue increase of 5%.

Strategic Alignment: Ensures scarce resources are allocated only to projects that contribute directly to corporate goals.

Digital Tools and System Integration



- Tool Complexity: The rise of specialized tools (PMIS, CRM, Accounting, Time Tracking) necessitates integration.
- Integration Challenge: Inefficient systems lead to wasted resources, manual reconciliation of budgets/schedules, and delayed reporting.
- Best Practice: Use integrated project management software (PMIS) that acts as a single source of truth, automatically syncing time logs (schedule data) with cost rates (budget data) for real-time EVM calculation.

The Impact of Generative AI on Estimation

- Use Case: Automatically analyzing a product requirements document (PRD) or user story backlog.
- AI Functionality:
 - Compares new requirements against thousands of historical, similar tasks.
 - Generates a probabilistic estimate (e.g., 80% chance of completion in 40-50 days).
 - Suggests resource profiles based on successful past projects.
- Caveat: AI provides a powerful *baseline*, but the final estimate must still be validated and adjusted by experienced human engineers (the "Human-in-the-Loop" validation).

Key Skills for Modern Project Managers



Skill Area	Traditional Focus	2020-2025 Focus
Technical	Gantt charts, WBS creation	Data <u>Visualization</u> , AI Tooling, Cloud Cost Management (FinOps)
Budget	Cost Variance, Budgeting	ROI/Value Metrics, Risk-Adjusted Forecasting
Soft Skills	Command and Control	Emotional Intelligence, Asynchronous Communication, Remote Team Leadership
Methodology	Waterfall/PMP	Hybrid Frameworks, Scaling Agile (SAFe, LeSS)

TASK NEXT WEEK

Case Study: Failed IT Project (Budget & Schedule Meltdown)

The "Phoenix" System Overhaul



Project Type: Large-Scale Enterprise Resource Planning (ERP) System Replacement (Internal Software Engineering Project).

Initial Goal: Replace 20-year-old legacy systems across Finance, HR, and Supply Chain with a single, integrated platform. Initial Scope: Comprehensive, fixed requirements set upfront.

Initial Budget: \$45 Million.

Initial Schedule: 24 Months.

TASK NEXT WEEK

Case Study: Failed IT Project (Budget & Schedule Meltdown)



Project Profile & Key Decisions

Parameter	Initial Plan (Waterfall/Predictive)	Reality Check
Methodology	Pure Waterfall (Requirements → Design → Build)	Hybrid (Requirements evolved constantly)
Key Resource	External Consulting Firm (Fixed Bid)	Consultant turnover was high; internal team lacked knowledge transfer.
Risk Register	Identified 50 minor risks.	Failed to predict regulatory changes that impacted core requirements.
Contingency	5% of budget (\$2.25M)	Depleted within the first 12 months due to design errors.

TASK NEXT WEEK

Case Study: Failed IT Project (Budget & Schedule Meltdown)



Problem 1: Uncontrolled Scope Creep and Volatility

The Issue: Requirements were "fixed" on paper, but essential, non-negotiable legal/regulatory updates (2022/2023) forced major mid-project design changes.

Problem 2: Waterfall Rigidity vs. Complexity

The Approach: The team attempted a strict, phase-gated Waterfall approach. Testing could only begin after *all* development was complete (end of Month 24).

Problem 3: Budget Overruns and Control Failure

TASK NEXT WEEK

Case Study: Failed IT Project (Budget & Schedule Meltdown)

Problem 3: Budget Overruns and Control Failure

Metric (Month 18)	Planned Value (PV)	Actual Cost (AC)	Earned Value (EV)	Variance/Index	Analysis
Value (Cost)	\$27M	\$30M	\$24M	CPI = 0.80	\$6M Over Budget. Only 80 cents of value earned for every dollar spent.
Value (Schedule)	\$27M	N/A	\$24M	SPI = 0.89	11% Behind Schedule. Work completed is only 89% of work planned.

The Outcome: The Estimated At Completion (EAC), based on the current poor performance, jumped from the initial \$45M to **\$75M**.

TASK NEXT WEEK

Case Study: Failed IT Project (Budget & Schedule Meltdown)

The Schedule Meltdown (Delays and Dependencies)

- Cause: Critical path dependency failure. A core financial module (Activity A) missed its deadline by 3 months.
- Impact: All downstream modules (Activities B, C, D) requiring Activity A's output were halted.
- Response: Management threw more resources (people, tools) at the problem.
 - The Myth: Adding resources to a late software project often makes it *later* (Brooks' Law). New staff required training, diverting existing resources.
- Actual Schedule: Initial 24 Months-> Final 42 Months (40% delay).

Summary



- AI & Automation:** Transitioning repetitive tasks (WBS creation, initial estimates) to AI for faster, data-driven baselines.
- Hybridization:** Predictive governance (Budget/Release) over Adaptive execution (Sprints/Features).
- Value-Driven:** Budget decisions are increasingly based on ROI, Time-to-Market (TTM), and Cost of Delay (CoD).
- Control:** Increased rigor through real-time metrics (CPI, SPI, Burn Rate) and digital tool integration for transparency.
- TCO Focus:** Project management scope is expanding to include life-cycle costs (especially cloud/operations).

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.